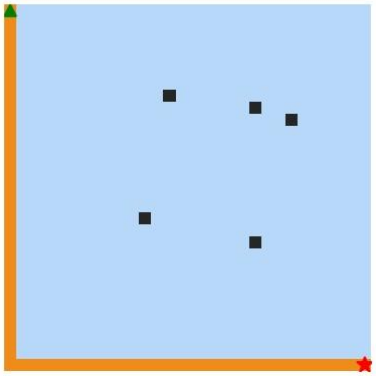
Assignment 2 – Program Output and Output Images (Tasks 1–7)

Task 1 – Greedy / A* / UCS on grid maps **Program output:**

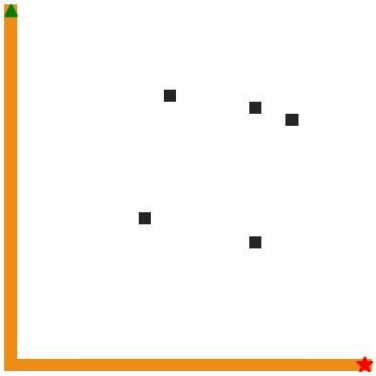
Map	Algo	Cost	Opt?	Expanded	Generated	MaxFront	ms
Map A - Open road	UCS	58	True	894	895	30	1.65
Map A - Open road	Greedy	58	True	58	115	57	0.15
Map A - Open road	A*	58	True	58	115	57	0.14
Map B - Dead-end trap	UCS	34	True	748	767	36	1.44
Map B - Dead-end trap	Greedy	38	False	62	121	59	0.22
Map B - Dead-end trap	A*	34	True	182	305	63	0.46
Map C - Serpentine maze (edge)	UCS	452	True	481	483	2	0.72
Map C - Serpentine maze (edge)	Greedy	452	True	466	483	17	0.81
Map C - Serpentine maze (edge)	A*	452	True	478	483	5	0.84

Cluttered random 30x30 maps (30 seeds per density; solvable instances only) density algo solved avg exp avg cost %optimal avg subopt% 0.10 UCS 30 810.3 58.0 100.0 0.00 0.10 Greedy 30 64.5 62.8 16.7 8.28 0.10 A* 30 93.1 58.0 100.0 0.00 0.20 UCS 26 718.3 58.0 100.0 0.00 0.20 Greedy 26 76.9 67.4 0.0 16.18 0.20 A* 26 111.7 58.0 100.0 0.00 0.25 UCS 22 664.0 58.3 100.0 0.00 0.25 Greedy 22 82.6 67.1 0.0 15.06 0.25 A* 22 122.3 58.3 100.0 0.00 0.30 UCS 16 604.4 59.2 100.0 0.00 0.30 Greedy 16 101.7 68.9 6.2 16.08 0.30 A* 16 144.9 59.2 100.0 0.00

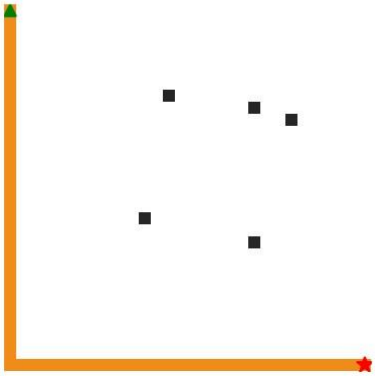
Map A - Open road
UCS: exp=894, cost=58



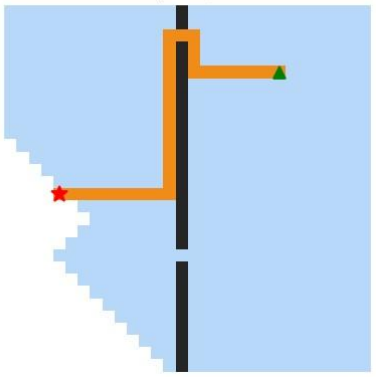
Map A - Open road
Greedy: exp=58, cost=58



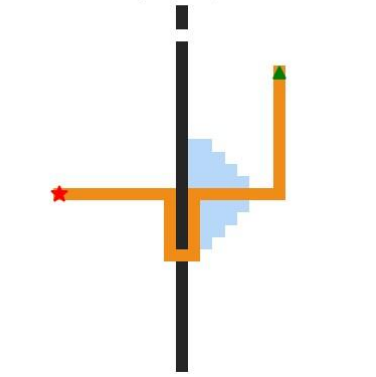
Map A - Open road
A*: exp=58, cost=58



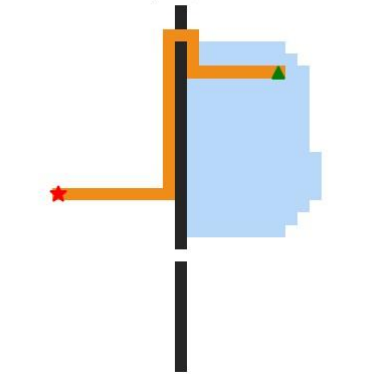
Map B - Dead-end trap
UCS: exp=748, cost=34



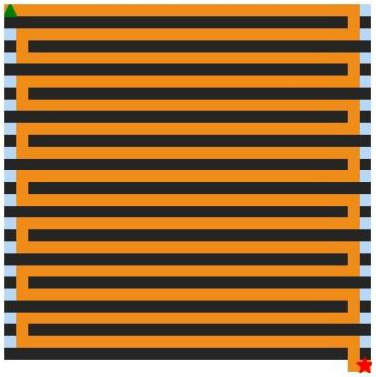
Map B - Dead-end trap
Greedy: exp=62, cost=38



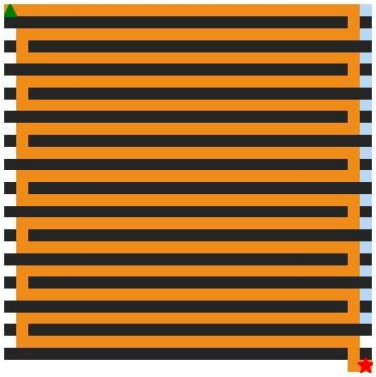
Map B - Dead-end trap
A*: exp=182, cost=34



Map C - Serpentine maze (edge)
UCS: exp=481, cost=452



Map C - Serpentine maze (edge)
Greedy: exp=466, cost=452



Map C - Serpentine maze (edge)
A*: exp=478, cost=452

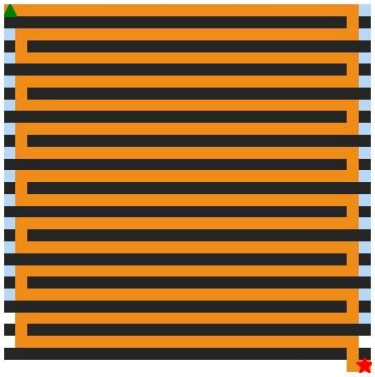


Figure: t1_maps

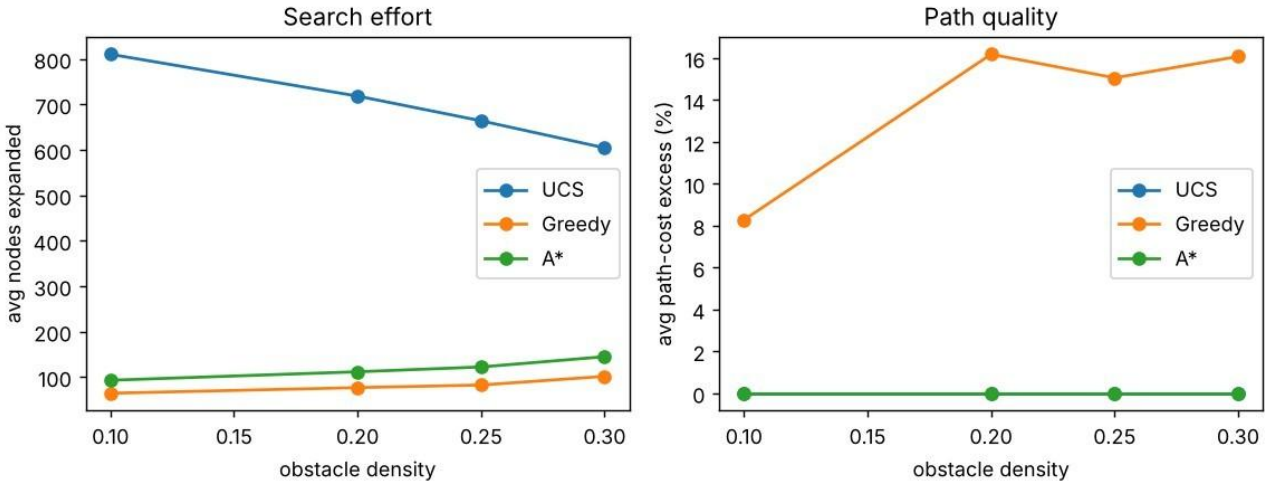


Figure: t1_clutter

Task 2 – Online search agent (DFS, LRTA*) **Program output:**

```
EXP A: target distance (31x31 maze, braid=0.2, 30 mazes)
target dist  opt cost    moves  revisits  comp. ratio  worst
ratio        0.10      19.9     20.5       0.3         1.04         1.53
0.25      49.5     56.6       2.5         1.14         2.32         0.50
99.6    159.1      27.2         1.75         11.15         0.75
149.1    239.0      39.7         1.66         5.84
       1.00     199.1    371.0      77.5         2.00         5.24

EXP B: maze size (far target = 75% of max distance, braid=0.2, 30 mazes)
  size free cells  opt cost    moves  revisits  comp.
ratio     11         49      27.9     29.7       0.7         1.07
21        201      84.1    125.6      19.9         1.63     31
454     149.1    239.0      39.7         1.66     41        807
227.4    328.8      41.6         1.48
   61       1817     407.3    854.1     194.2         2.12

EXP C: dead-end density (31x31, target 75%, 30
mazes)  braid dead-end dens.  opt cost    moves
revisits  comp. ratio   0.00         0.054     213.3    266.6
26.7         1.26   0.25         0.040     147.3    235.0
39.9         1.65   0.50         0.027     105.4    215.0
41.8         2.12   0.75         0.014      81.4    187.8
37.8         2.30
  1.00         0.000      65.9    131.6      17.0         1.98

EXP D: unreachable target (target cell sealed off; agent must exhaust the reachable region)
  size free cells  moves to give up  revisits
moves/cell     11         47            68.3      34.1
1.46     21        198           363.9     181.9        1.84
31       451           822.1     411.1        1.82     41
804           1500.8     750.4        1.87

EXP E: LRTA* over repeated missions (31x31, braid 0.2, target 75%, 20 mazes, 60 trials) - avg moves
per trial
trial:              1       2       3       5       10      20      30      45      60 avg moves:
2631.5   385.6   435.1   408.1   755.3   366.3   305.2   254.6   207.4
avg optimal cost = 152.0; avg online-DFS moves (every mission) = 230.7
competitive ratio LRTA* at those trials: 17.31 2.54 2.86 2.68 4.97 2.41
2.01 1.68 1.36

CHANGED CONDITION: cumulative travel for N missions per facility (31x31 maze, avg of 20 mazes)
 N missions   online DFS every time   map once + plan  LRTA* learning
```

1	231	905	2632	2
461	1057	3017	5	1154
1513	4262	10	2307	2273
6449	50	11535	8353	18561
200	46140	31153	49972	

(one full-exploration run costs 905 moves; x20 facilities x20 rovers multiplies every column by 20)

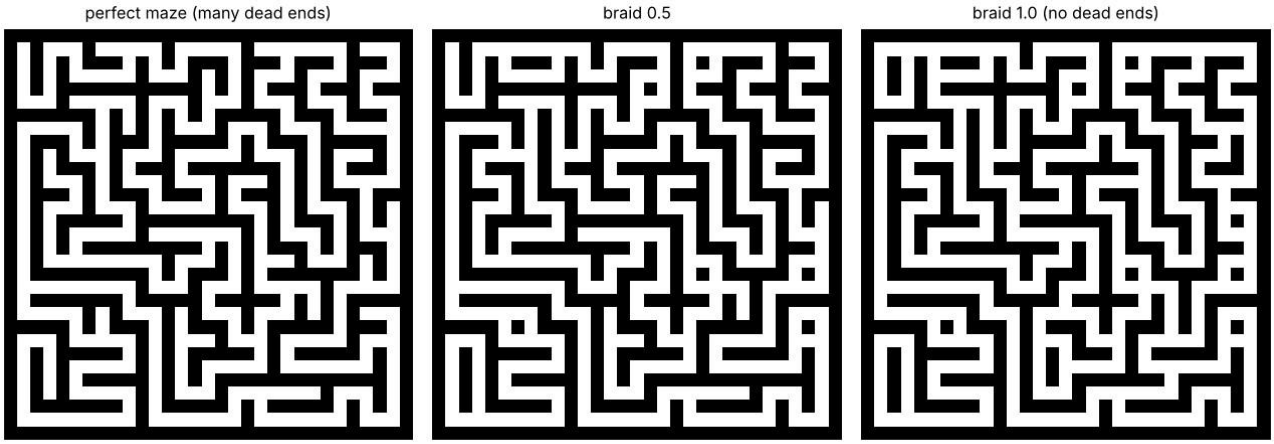


Figure: t2_mazes

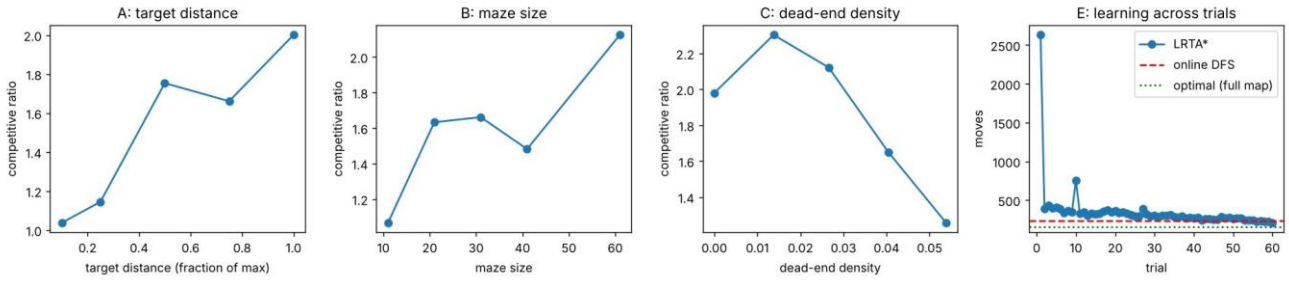


Figure: t2_charts

Task 3 – Alpha-beta pruning **Program**

output:

b=4, d=6: minimax visits b^d = 4096 leaves, 5461 nodes in total
PREDICTIONS (written before running):
T1 best-ordered : leaves ~ b^ceil(d/2)+b^floor(d/2)-1 = 127
T2 engineered worst: leaves = b^d = 4096 (zero cutoffs; best move is always LAST at every node)
T2b merely sorted worst-first by local value: slightly below b^d (deep bounds from ancestors still cut a few branches)
T3a random order: between the two, ~ b^(3d/4) = 512 leaves on average
T3b equal leaves: ties trigger cutoffs (alpha>=beta) -> behaves like best-ordered: ~127 leaves

Tree	minimax leaves	AB leaves	AB nodes	cutoffs	% leaves saved
T1 best-ordered (many cutoffs)	4096	127	268	99	96.9
T2 engineered worst (no cutoffs)	4096	4096	5461	0	0.0
T2b random tree sorted worst-first	4096	3663	4970	670	10.6
T3a random order	4096	1092	1664	326	73.3
T3b all leaves equal	4096	127	268	99	96.9

T3a repeated on 200 random trees: mean AB leaves = 923 (min 476, max 1342); prediction ~512

Depth sweep, b=4: alpha-beta leaves visited

d	minimax b^d	best-ordered	formula	random(avg of 20)	worst-ordered
2	16	7	7	12	16
3	64	19	19	40	64
4	256	31	31	118	256
5	1024	79	79	341	1024
6	4096	127	127	867	4096
7	16384	319	319	2769	16384
8	65536	-	511	-	-



Figure: t3_charts

Task 4 – Depth-limited minimax (Connect-Four) **Program**

output:

Theory: nodes N(b,d) = (b^(d+1)-1)/(b-1); leaves(terminal evaluations) = b^d (valid while no win/full-board occurs)

depth d theory nodes measured nodes theory leaves measured evals time (s) 1 8 8 7 7 0.000
2 57 57 49 49 0.002 3 400 400 343 343 0.011 4 2801 2801 2401 2401 0.080 5 19608 19608 16807 16807 0.572 6 137257 137257 117649 117649 4.115
7 960800 960793 823543 823536 30.320

Board size effect (rows=6, depth=5, empty board) board b theory nodes measured evals time (s) 6x4 4 1365 1365 1024 0.014 6x5 5 3906 3906 3125 0.064 6x6 6 9331 9331 7776 0.211 6x7 7 19608 19608 16807 0.570 6x8 8 37449 37449 32768 1.330

Playing strength vs RANDOM player (6x7, 20 games: 10 AI-first, 10 AI-second) depth wins draws losses win rate AI moves nodes/move evals/move s/move 1 20 0 0 1.00 101 7 7 0.000 2 20 0 0 1.00 122 54 47 0.002 3 20 0 0 1.00 124 371 319 0.011 4 20 0 0 1.00 119 2511 2149 0.078

Playing strength vs a DEPTH-1 minimax opponent (20 games each, deeper AI alternates first/second) AI depth wins draws losses win rate
1 10 0 10 0.50
2 10 0 10 0.50 3 20 0 0 1.00

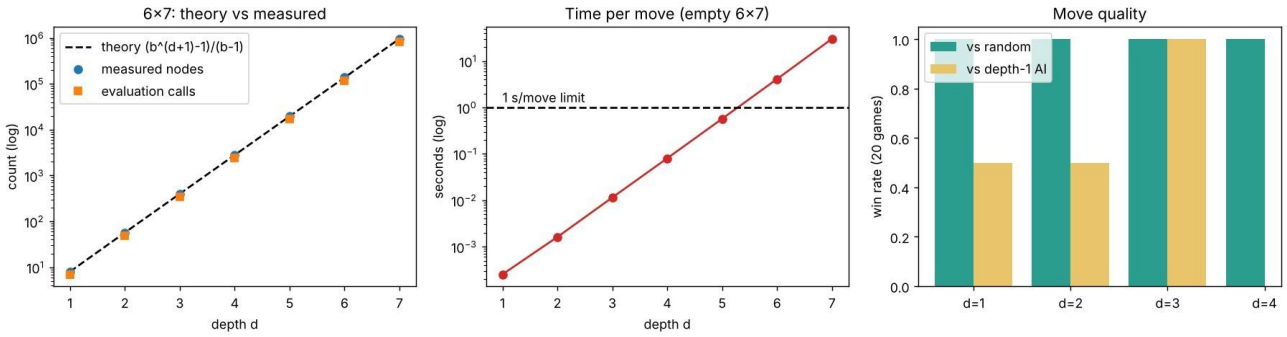


Figure: t4_charts

Task 5 – Constraint propagation (AC-3) for timetabling

Program output:

PSEUDOCODE (student-derived from the support principle) revise(xi, xj): for each x in D[xi]: if no y in D[xj] with compatible(x,y): remove x; return removed? AC(CSP): Q = all directed arcs (xi,xj) for each conflicting pair; while Q not empty: (xi,xj)=pop(Q)
if revise(xi,xj): if D[xi] empty -> return FAIL (domain wipe-out = no solution) else push (xk,xi) for every neighbour xk of xi with xk != xj return OK
SOLVE: if AC fails -> no solution; if every |D|=1 -> solution; else pick variable, try each value v: copy domains, set D[x]={v}, run AC with Q = {(xk,x)}, recurse if no wipe-out, else undo (backtrack)

AC removes 1 from D[Physics] (no support in Maths)
AC removes 2 from D[Chem] (no support in Physics)
AC removes 1 from D[Chem] (no support in Maths)
AC removes 3 from D[Bio] (no support in Chem)
AC removes 2 from D[CS] (no support in Bio)
AC removes 3 from D[CS] (no support in Chem)
Instance 1 final domains: {'Maths': [1], 'Physics': [2], 'Chem': [3], 'Bio': [2], 'CS': [1]} | checks = 28 | total domain size trace: [11, 10, 9, 8, 7, 6, 5]
Instance 1: all domains became singletons -> solution read off with search nodes = 5 {'Maths': 1, 'Physics': 2, 'Chem':
3, 'Bio': 2, 'CS': 1}
Instance 1b (triangle, A fixed to slot 1, B,C in {1,2}): solution = None | wipe-outs = 1 | search nodes = 0 -> failure
detected by propagation alone plain backtracking on the same
instance: nodes = 5 backtracks = 2 checks = 5

Instance 2 (K4 with 3 slots: 4 mutually clashing exams): AC alone reduced nothing: True | solution: None | search nodes with MAC = 9 wipe-outs = 6

SCALING: random conflict graphs, k=5 slots, edge prob p=0.30 (near the hard region), 10 instances per n; node cap 30000

n	method	avg nodes	avg backtracks	avg checks	avg wipeouts	capped %solved
10	BT	17	0	22	0.0	0	100
10	BT+AC3	10	0	290	0.0	0	100
15	BT	31	0	60	0.0	0	100
15	BT+AC3	15	0	809	0.0	0	100
20	BT	3050	601	10322	0.0	1	90
20	BT+AC3	41	21	3285	13.1	0	100
25	BT	4003	788	14474	0.0	1	90
25	BT+AC3	38	12	3278	8.1	0	100
30	BT	23962	4780	86976	0.0	7	30
30	BT+AC3	3455	3427	522358	2026.3	1	90
40	BT	30004	5989	110714	0.0	10	0	40
40	BT+AC3	27153	27142	7827875	17764.2	9	10

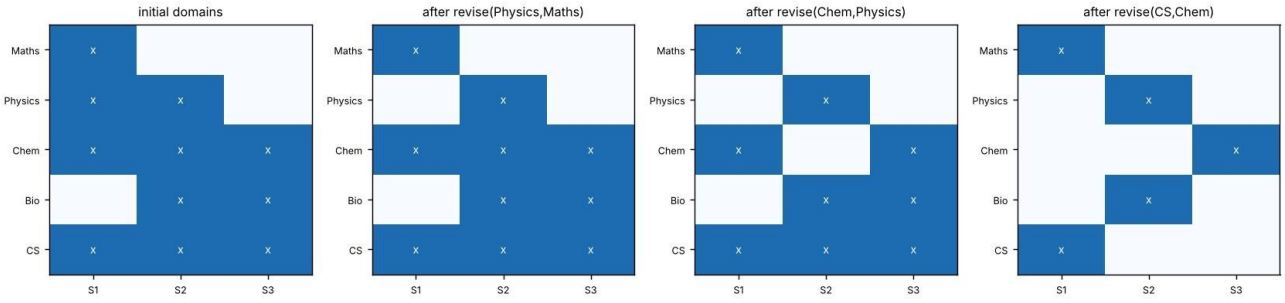


Figure: t5_inst1_domains

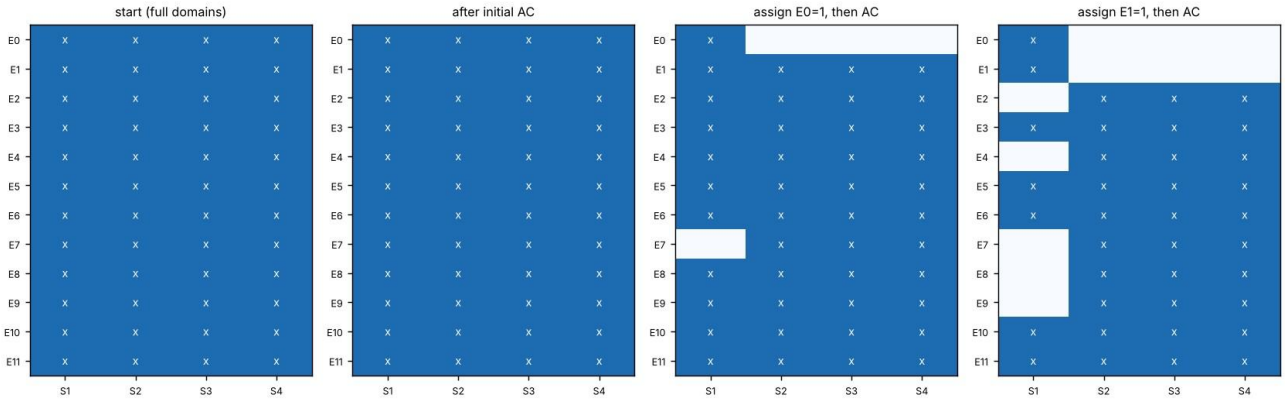


Figure: t5_search_domains

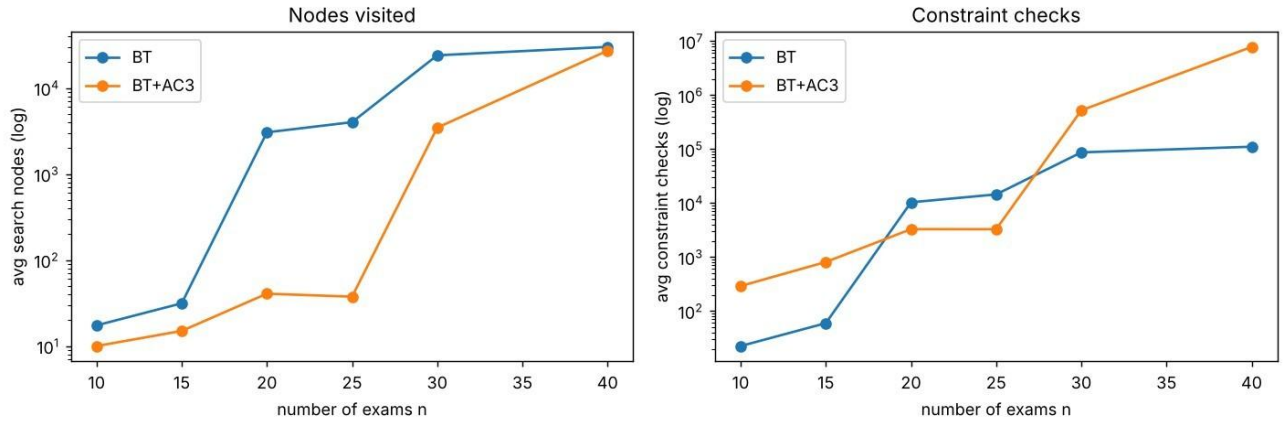


Figure: t5_scaling

Task 6 – Exhaustive vs local search (delivery tour) **Program output:**

CORRECTNESS 1: depot + 4 customers at hand-checkable coordinates
independent check via itertools.permutations: 24 tours; best = 14.4721 [0, 1, 2, 3, 4]; worst = 19.6569 my exhaustive search : tours enumerated = 24, best = 14.4721 [0, 1, 2, 3, 4] -> matches: True

CORRECTNESS 2: n=10 points on a circle (radius 50): optimum must be the perimeter 2*n*R*sin(pi/n) (visiting in circular order) exhaustive = 309.016994, analytic perimeter = 309.016994, match = True, tours enumerated = 362880 (= 9! = 362880)

GROWTH: tours enumerated (prediction (n-1)!) vs measured, random instances n predicted (n-1)! measured time (s) time ratio to prev predicted ratio (n-1) 5 24 24 0.000 nan nan 6 120 120 0.000 3.21 5 7 720 720 0.000 5.99 6 8 5040 5040 0.003 7.34 7 9 40320 40320 0.024 8.56 8 10 362880 362880 0.219 9.14 9 11 3628800 3628800 2.323 10.61 10

LOCAL SEARCH vs exhaustive optimum: 20 runs each (independent random seeds), 2-opt hill climbing
 n restarts mean gap % max gap % runs hitting opt mean time (ms) 8 1 2.35 7.83 14/20 0.02 8 5 0.00 0.00 20/20 0.05 8 20 0.00 0.00 20/20 0.20 9 1 3.04 6.75 11/20 0.01 9 5 0.00 0.00 20/20 0.06 9 20 0.00 0.00 20/20 0.24 10 1 2.08 8.31 15/20 0.02 10 5 0.00 0.00 20/20 0.08 10 20 0.00 0.00 20/20 0.33 11 1 1.34 8.91 17/20 0.02 11 5 0.00 0.00 20/20 0.10 11 20 0.00 0.00 20/20 0.38

Larger sizes (optimum unknown): gap of 5-restart 2-opt relative to best tour found in 200 restarts, 20 runs
 n= 15 mean gap 0.00% max 0.00% mean time 0.2 ms n= 20 mean gap 0.04% max 0.25% mean time 0.3 ms n= 30 mean gap 0.98% max 2.87% mean time 0.8 ms n= 50 mean gap 2.23% max 6.13% mean time 2.3 ms n=100 mean gap 2.25% max 5.21% mean time 11.1 ms

Exhaustive cost per complete tour ~ 0.64 microseconds (measured at n=11)
 n tours est. time 10 362880 0.23 s 11 3628800 2.32 s 12 39916800 25.55 s 13 479001600 306.63 s 14 6227020800 1.1 h 15 87178291200 15.5 h 20

121645100408832000 2.47e+03
years

Example trapped run: local minimum 389.64 vs optimum 357.78 (+8.9%)

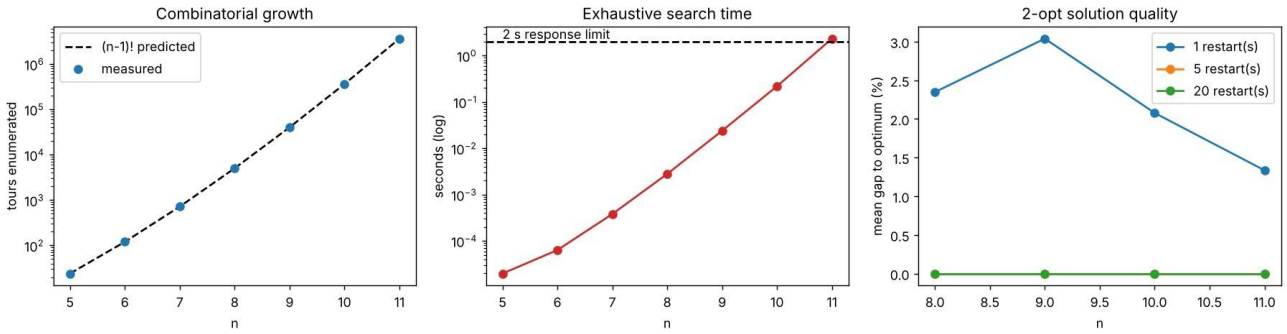


Figure: t6_charts

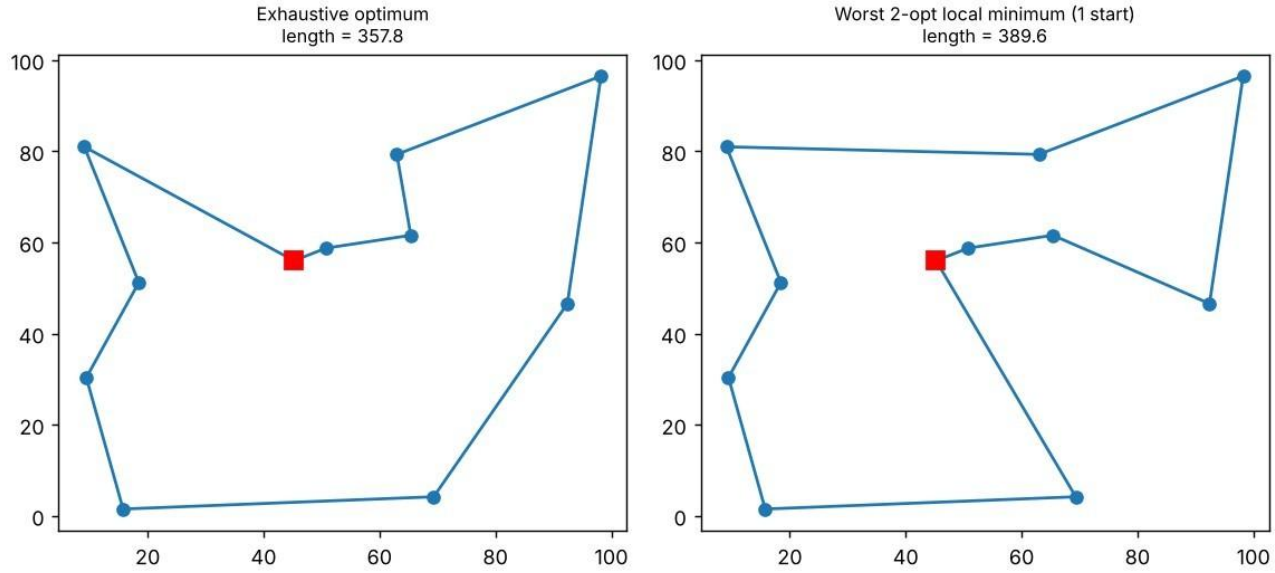


Figure: t6_tours

Task 7 – Redesign: branch-and-bound tour search **Program**

output:

PREDICTION (written before running): V0 expands ~ e*(n-1)! nodes. With pruning I expect nodes to grow far more slowly
(roughly c^n, c<n) on spread-out geometric instances; V4 (good incumbent from the start) should be the cheapest; on nearuniform distances the bound is weak and the gain should almost vanish. Optimal cost must be identical for all variants.

A) Ablation, uniform random points, n=10 (10 instances): nodes expanded / tours completed / bound evals / time

variant	nodes	leaves	bound evals	time (ms)	same optimum
V0 original exhaustive	986410	362880	0	223.7	True
V1 prefix-cost pruning	74289	1323	0	35.2	True
V2 + lower bound	12927	483	35927	8.1	True
V3 + nearest-first	9598	343	27960	6.2	True
V4 + 2-opt seeded incumbent	8888	240	26784	5.8	True

B) Scaling with n, uniform random points, 10 instances each (nodes expanded; V0 measured for n<=11, formula beyond)

n	V0 original	V1	V2	V3	V4	V4 time ms	speed-up
V0/V4	6	326	207	119	96	96	0.1	3.4
8	13700	3408	1100	842	811	0.5	16.9
10	986410	74289	12927	9598	8888	5.8	111.0
11	9864101	355176	45256	32718	29536	20.9	334.0
12	108505112	1828836	133469	90905	84157	66.6	1289.3
13	1302061345	nan	484415	328782	303742	264.2	4286.7
14	16926797486	nan	1113469	652290	597833	573.3	28313.6
15	236975164805	nan	nan	2356148	2243052	2245.3	105648.6

check of node formula for V0 at n=9: counted by instrumented run vs formula counted = 109601, formula = 109601

C) Instance structure (n=12, 10 instances each; V4 vs V0 formula)

instance type	V0 nodes	V4 nodes	reduction x	V4 time ms
uniform random	108505112	57333	1892.5	47.3
clustered (3 clusters)	108505112	388623	279.2	247.7
near-uniform distances	108505112	12476	8697.2	10.7
twin pairs (weak bound)	108505112	577547	187.9	413.3

E) Small n: wall-clock V0 vs V4 (10 instances, ms) - where the original is still fine

n	V0 ms	V4 ms
4	0.005	0.060
5	0.013	0.049
6	0.062	

0.079 7 0.370
0.187 8 2.716
0.503

Near-uniform case at n=10, where V0 is cheap: V0 vs V4 wall-clock (V4 pays for bound evaluation on every child)

V0: nodes=986410, 214.2 ms | V4: nodes=2025, bound evals=6982, 1.5 ms

D) Largest n solved to optimality within the 2 s budget by V4 (uniform random, 3 instances; worst instance reported)

n=13: mean 0.38 s, worst 0.53 s

n=14: mean 0.93 s, worst 1.37 s

n=15: mean 4.28 s, worst 9.00 s

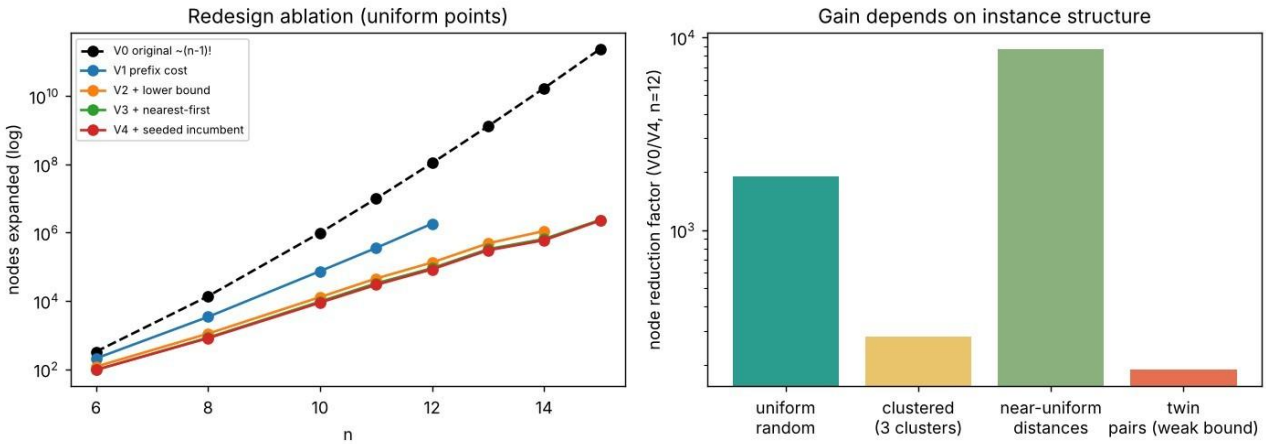


Figure: t7_charts